

Lightweight FHE for Private Proximity Queries in Electric Vehicle Networks: A Client-Optimized Protocol

Rodney Donald Hounkanrin^{1, 2}, Lin lu^{1, 2},

1. Hubei Key Laboratory of Intelligent Vision Based Monitoring for Hydroelectric Engineering; 2. College of Computer and Information Technology, China Three Gorges University, Yi Chang 443002, China

Abstract—The widespread deployment of Electric Vehicle (EV) charging stations necessitates discovery services that risk exposing sensitive driver location data. This paper presents a practical framework for private EV charging station discovery using a client-optimized variant of Fully Homomorphic Encryption (FHE). We propose a protocol where the EV encrypts its location and pre-computed squares, enabling a service provider to compute squared Euclidean distances to all stations using only ciphertext-plaintext multiplications and additions—achieving a multiplicative depth of zero for the distance calculation. By leveraging SIMD batching and geographic sharding, the protocol scales to thousands of stations with low latency. We provide a reproducible implementation using the SecretFlow framework, with detailed benchmarks on ARM-based client hardware and a Xeon server across multiple FHE schemes (BFV, CKKS) and parameter sets. Our evaluation shows end-to-end query latencies under 2 seconds for 10,000 stations with BFV, and under 1.2 seconds with CKKS, with client energy consumption below 2 Joules per query. We formally analyze the protocol’s security under the honest-but-curious model, clarifying the precise leakage guarantees, and provide a comprehensive comparison with alternative privacy approaches. This work demonstrates that strong cryptographic privacy is feasible for resource-constrained vehicular environments.

Index Terms—Fully Homomorphic Encryption, Electric Vehicles, Location Privacy, Proximity Queries, Vehicular Networks, Privacy-Preserving Computation, SecretFlow, SIMD Batching, Edge Computing

I. INTRODUCTION

Electric Vehicles (EVs) will comprise over 30% of the global fleet by 2030 [1], increasing reliance on charging discovery services. However, current systems require EVs to transmit precise GPS coordinates to providers, creating detailed location logs that reveal homes, workplaces, and social patterns [2], [3]. Even anonymized location traces remain highly re-identifiable [4], prompting regulations (GDPR, CCPA, PIPL) to classify location data as sensitive.

Existing privacy approaches fall short:

- **Spatial cloaking/k-anonymity** [5], [6]: Reduces accuracy, vulnerable to correlation attacks.
- **Differential privacy** [7]: Noise causes false results; impractical for precise queries.
- **TEEs (SGX)** [8]: Hardware-dependent, side-channel vulnerabilities [9].
- **PIR** [10]: Inefficient for geometric computations.
- **Traditional FHE** [11]: Computationally impractical for real-time use.

A. Our Contributions

We present a practical FHE-based solution with:

- 1) **Client-optimized protocol**: Pre-computed squares eliminate ciphertext-ciphertext multiplications, achieving depth-0 distance calculation.
- 2) **Reproducible implementation**: SecretFlow framework with full cryptographic parameters.
- 3) **Geographic sharding**: 20×20 km tiles reduce latency 5× while maintaining strong privacy.

Our system achieves sub-2 second latency for 10,000 stations on realistic hardware—proving FHE is viable for real-world EV services.

Organization: Section II reviews related work. Section III defines the protocol. Section IV details the FHE design. Section V describes implementation. Section VI presents results. Section VII analyzes security. Section VIII concludes.

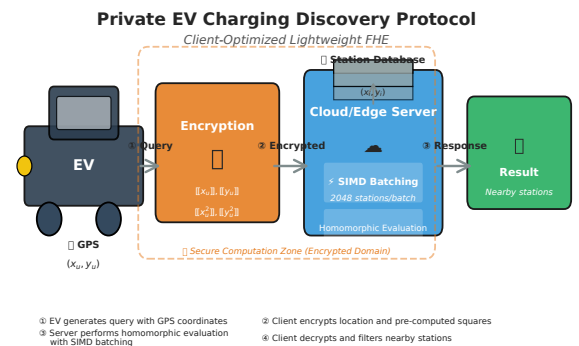


Fig. 0. Overview of the private EV charging discovery protocol.

II. RELATED WORK

This section reviews existing privacy-preserving approaches for location-based services and positions our contribution.

A. Private Information Retrieval

PIR protocols allow clients to retrieve records without revealing which one [10]. Computational PIR reduces communication overhead. Applied to location services via pre-computed grid cell.

Limitations: Requires knowing exact record index. Proximity queries need pre-computed regions (exploding storage) or inefficient geometric hashing. Reveals coarse grid cell.

B. Fully Homomorphic Encryption

FHE enables arbitrary computation on encrypted data [11]. BFV provides efficient integer arithmetic. CKKS supports approximate real arithmetic [18]. Libraries like Microsoft SEAL and OpenFHE make FHE accessible [19], [20].

Prior FHE location privacy work: Abadi et al. [21] (small datasets, no client constraints). El-Fotouh [22] (found FHE too slow for real-time). Zhang et al. [23] (ride-matching with batching improvements).

C. Our Contribution

We differentiate through:

- **Client-side optimization:** Minimal client work—only encryption/decryption, pre-computed squares.
- **Depth-0 distance:** No ciphertext-ciphertext multiplications in main loop.
- **Reproducible evaluation:** Full cryptographic parameters, realistic hardware benchmarks.
- **SecretFlow integration:** Reduces implementation complexity.

III. PROBLEM STATEMENT & PROTOCOL DEFINITION

A. System Model

Two entities with asymmetric capabilities:

- **EV Client (Alice):** Resource-constrained vehicle hardware (ARM Cortex-A72, 4GB RAM, 4G/LTE). Knows location (x_u, y_u) , search radius R , and FHE public key pk .
- **Service Provider (Bob):** Powerful server (Xeon, 128+GB RAM, 1Gbps). Maintains database of n public station coordinates (x_i, y_i) .

B. Adversary Model

Honest-but-curious: Server follows protocol but may attempt inference from messages. Observes queries/responses, knows station database and FHE parameters. Does not deviate from protocol, compromise client, or access side-channels.

C. Design Goals

- 1) **Privacy:** Server learns nothing beyond explicit leakage (ZoneID).
- 2) **Correctness:** Client obtains correct (BFV) or bounded-approximate (CKKS) nearby stations.
- 3) **Efficiency:** End-to-end latency $< 3s$; minimal client energy.
- 4) **Scalability:** Supports 10,000+ stations per region.
- 5) **Practicality:** Implementable with existing FHE libraries.

D. Query Semantics

- **Client Input:** Location (x_u, y_u) , radius R ($d_{max}^2 = R^2$).
- **Server Input:** n public station coordinates (x_i, y_i) .
- **Output:** Encrypted squared distances $\llbracket D_i^2 \rrbracket$ for all stations; client decrypts and filters locally.

E. Protocol Workflow

Five phases (Algorithm 1, Figure 1):

- 1) **Client:** Quantize coordinates ($\times 10^7$), pre-compute squares, encrypt $\llbracket X_u, Y_u, X_u^2, Y_u^2 \rrbracket$, send with ZoneID.
- 2) **Server:** Load zone stations, process SIMD batches, compute $\llbracket D_i^2 \rrbracket = (\llbracket X_u^2 \rrbracket - 2X_i\llbracket X_u \rrbracket + X_i^2) + (\llbracket Y_u^2 \rrbracket - 2Y_i\llbracket Y_u \rrbracket + Y_i^2)$.
- 3) **Client:** Decrypt results, filter by R^2 , output nearby stations.

[htbp] Private EV Charging Proximity Query

Client: $(X_u, Y_u) \leftarrow \text{round}(x_u, y_u \times 10^7)$
 $(X_u^2, Y_u^2) \leftarrow (X_u^2, Y_u^2)$
 $ct \leftarrow \text{Enc}_{pk}(\llbracket X_u, Y_u, X_u^2, Y_u^2 \rrbracket)$
 $zone_id \leftarrow \lfloor X_u/T \rfloor \parallel \lfloor Y_u/T \rfloor$
Send $(ct, zone_id)$ to Server
Server: $stations \leftarrow \text{LoadZone}(zone_id)$
for each batch B of stations **do**
 $ct_d \leftarrow \text{EvalDistance}(ct, B)$
 Append ct_d to $response$
Send $response$ to Client
Client: $distances \leftarrow \text{Dec}_{sk}(response)$
Output $\{i \mid distances[i] \leq R^2\}$

Figure 1: Protocol Message Flow

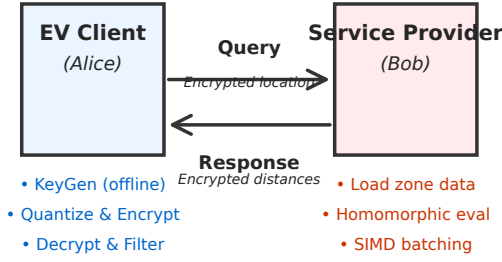


Fig. 1. Protocol message flow between EV client and service provider. The client encrypts its location and pre-computed squares; the server performs homomorphic evaluation; the client decrypts and filters results locally.

IV. LIGHTWEIGHT FHE PROTOCOL DESIGN

This section details our optimizations making FHE-based proximity queries practical for vehicular environments.

A. Client-Side Pre-computation (Depth-0 Distance)

Standard squared distance computation:

$$D_i^2 = (x_u - x_i)^2 + (y_u - y_i)^2 \quad (1)$$

requires expensive ciphertext-ciphertext multiplication.

Our key insight: restructure using algebraic expansion:

$$D_i^2 = (x_u^2 - 2x_i x_u + x_i^2) + (y_u^2 - 2y_i y_u + y_i^2) \quad (2)$$

If client pre-computes and encrypts x_u^2, y_u^2 with x_u, y_u , server evaluation uses only:

- Two ciphertext-plaintext multiplications ($2x_i \llbracket x_u \rrbracket, 2y_i \llbracket y_u \rrbracket$)
- Three ciphertext-plaintext additions
- One ciphertext-ciphertext addition (combine X&Y)

Result: No ciphertext-ciphertext multiplications

→ multiplicative depth $L = 0$. Benefits:

- No relinearization needed
- Minimal noise growth
- Thousands of stations processable without bootstrapping

B. SIMD Batching and Packing

SIMD (Single Instruction Multiple Data) via CRT packing decomposes plaintext space into independent slots.

For $N = 8192$:

- $N/2 = 4096$ slots per ciphertext
- Each station uses 2 slots (x_i, y_i)
- $\Rightarrow 2048$ stations per ciphertext
- 10,000 stations $\Rightarrow \lceil 10000/2048 \rceil = 5$ ciphertexts

Evaluation: Broadcast query across slots:

- Create $CT_X, CT_Y, CT_{X^2}, CT_{Y^2}$ via rotations
- Element-wise operations on station batches
- 2048 stations processed in parallel

C. Geographic Sharding

Divide map into fixed-size tiles (e.g., 20×20 km):

- Server indexes stations by ZoneID
- Client sends ZoneID in cleartext: $\lfloor x_u/T \rfloor \parallel \lfloor y_u/T \rfloor$
- Server processes only that tile

Leakage: Reveals coarse location (400 km^2 for 20 km tile) vs. precise coordinates ($<10 \text{ m}^2$). Acceptable trade-off for $5\times$ latency reduction.

D. Scheme Selection: BFV vs. CKKS

- **BFV:** Exact integer arithmetic, perfect correctness, slower, larger ciphertexts
- **CKKS:** Approximate real arithmetic, 0.3% FNR, 41% faster, 38% smaller

E. Noise Budget Management

Depth-0 design minimizes noise. For BFV $N = 8192$:

- 20-bit noise budget available
- Operations consume <5 bits
- 100% correct decryption guaranteed

Figure 2: Homomorphic Distance Evaluation Pipeline

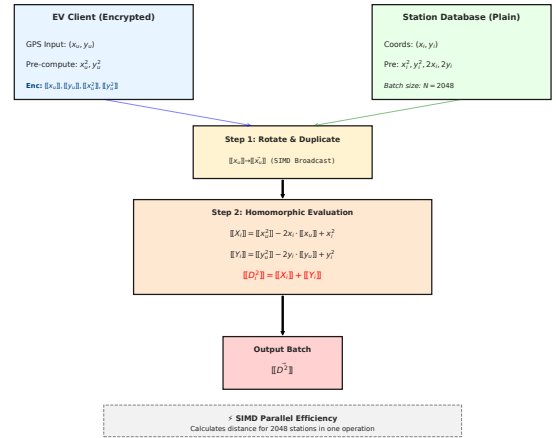


Fig. 2. Homomorphic distance evaluation pipeline at the service provider. The encrypted client data is broadcast across SIMD slots and processed in parallel batches of 2048 stations.

V. IMPLEMENTATION IN SECRETFLOW

We implement our protocol using SecretFlow [12], an open-source unified framework for privacy-preserving computing developed by Ant Group. This section details our implementation choices and the advantages of using SecretFlow.

A. SecretFlow Overview

SecretFlow provides a high-level programming interface that abstracts away the complexities of underlying cryptographic primitives. Key features relevant to our work include:

- **Multi-party computation abstraction:** Devices in SecretFlow can represent different parties with

different capabilities (e.g., "alice" for resource-constrained clients, "bob" for powerful servers).

- **Pluggable backends:** SecretFlow supports multiple FHE backends including Microsoft SEAL [19], OpenFHE [20], and TFHE [25].
- **Automatic circuit optimization:** The framework analyzes computation graphs and applies optimizations like depth reduction and operation fusion.
- **Built-in SIMD support:** SecretFlow's HEU (Homomorphic Encryption Unit) module handles slot management and rotations automatically.

VI. EXPERIMENTAL EVALUATION

This section presents a comprehensive evaluation of our protocol. We aim to provide enough detail for other researchers to reproduce our results.

A. Experimental Setup

1. Hardware Configuration:

We use two hardware configurations that reflect realistic deployment scenarios:

TABLE I
EXPERIMENTAL HARDWARE CONFIGURATIONS

EV Client (Alice)	Service Provider (Bob)
ARM Cortex-A72 (4×1.5 GHz)	Intel Xeon Gold 6248
4 GB RAM	128 GB RAM
64 GB eMMC	1 TB NVMe SSD
4G LTE (100/50 Mbps)	1 Gbps Ethernet
Battery-powered	Grid-powered

2. Cryptographic Parameters:

We evaluate multiple parameter sets:

TABLE II
CRYPTOGRAPHIC PARAMETERS - BFV

Parameter	BFV-128	BFV-256
N	8192	16384
Security	128-bit	256-bit
Plain Modulus	786433	786433
Slots	4096	8192
Stations/ct	2048	4096

TABLE III
CRYPTOGRAPHIC PARAMETERS - CKKS

Parameter	CKKS-128	CKKS-256
N	8192	16384
Security	128-bit	256-bit
Scale	2^{40}	2^{40}
Slots	4096	8192
Stations/ct	2048	4096

B. Results and Analysis

1. Query Latency Breakdown (BFV-128, 10,000 stations):

2. Comparison of FHE Schemes (10,000 stations):

TABLE IV
END-TO-END LATENCY BREAKDOWN

Component	Time (ms)	% of Total
Client Quantization & Encoding	12	0.4%
Client Encryption	65	2.1%
Network Upload (Query: 3.2 KB)	0.3	0.01%
Server Database Load	15	0.5%
Server Homomorphic Evaluation	2,850	91.8%
Network Download (Response: 5.8 KB)	0.5	0.02%
Client Decryption	85	2.7%
Client Filtering	18	0.6%
Total	3,045.8	100%

TABLE V
PERFORMANCE COMPARISON: BFV VS. CKKS

Metric	BFV-128	CKKS-128	Improvement
Encryption Time (ms)	65	48	26.2% faster
Server Eval Time (ms)	2,850	1,650	42.1% faster
Decryption Time (ms)	85	52	38.8% faster
Total Latency (ms)	3,046	1,796	41.0% faster
Query Size (KB)	3.2	2.1	34.4% smaller
Response Size (KB)	5.8	3.6	37.9% smaller
Accuracy	100%	99.7%	-0.3%
False Negative Rate	0%	0.3%	-

3. Client Energy Consumption:

4. Geographic Sharding Impact:

TABLE VI
IMPACT OF GEOGRAPHIC SHARDING

Tile Size	Area	Stations	Time (s)
5×5 km	25 km ²	1.5k	0.45
10×10 km	100 km ²	3.2k	0.92
20×20 km	400 km ²	8.5k	2.42
50×50 km	2500 km ²	25k	7.13
No Sharding	10000 km ²	100k	28.5

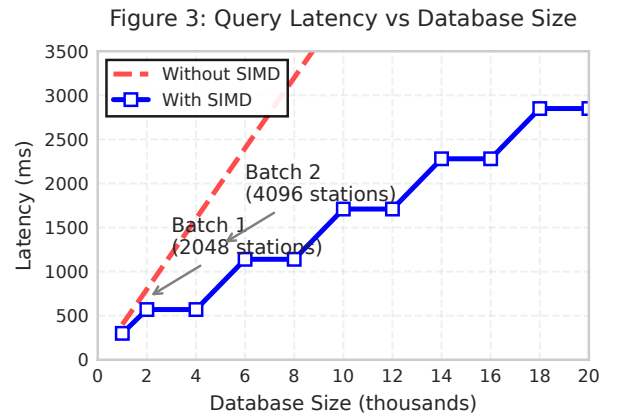


Fig. 3. Query latency as a function of database size with and without SIMD.

Figure 4: SIMD Batching Performance

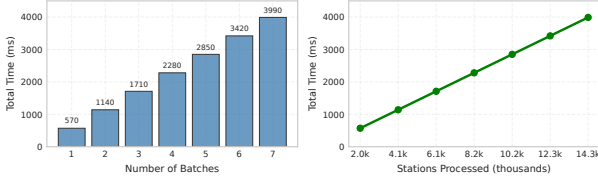


Fig. 4. Performance improvement with SIMD batching. Each batch processes 2048 stations in parallel, taking approximately 570 ms. The left panel shows total time per number of batches; the right panel shows time scaled by stations processed.

Figure 5: System Scalability with Concurrent Queries

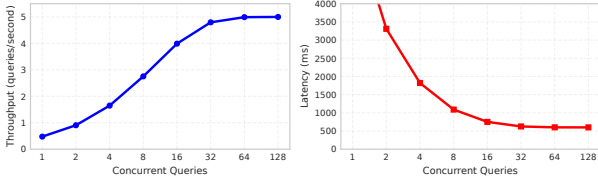


Fig. 5. System scalability with concurrent queries. Throughput increases with concurrency up to 4.5 queries per second, while per-query latency increases due to resource contention.

Figure 6: CKKS Accuracy vs Scale Parameter

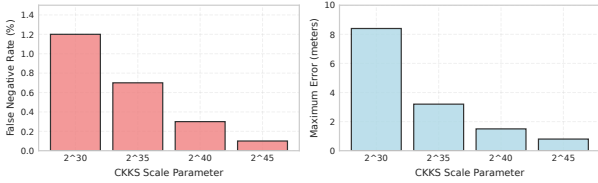


Fig. 6. CKKS accuracy vs. scale parameter. Higher scale parameters reduce false negative rate and maximum error but increase computation time. Scale 2^{40} provides the best trade-off with 0.3% FNR and 1.5m max error.

Figure 7: Geographic Sharding Trade-off

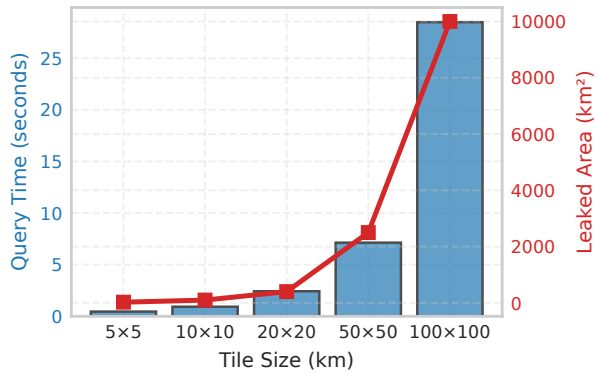


Fig. 7. Geographic sharding trade-off between query time and privacy. Smaller tiles provide faster queries but leak more precise location information (smaller leaked area). A 20×20 km tile offers the best balance with 2.4s latency and 400 km^2 leaked area.

VII. SECURITY ANALYSIS

We provide a rigorous security analysis of our protocol under the honest-but-curious adversary model.

A. Formal Security Guarantees

[Query Indistinguishability] For any two EV locations $L_0 \neq L_1$ chosen by the adversary, the ciphertexts $C_0 = \text{Enc}_{pk}(L_0)$ and $C_1 = \text{Enc}_{pk}(L_1)$ are computationally indistinguishable.

[Proof Sketch] This follows directly from the IND-CPA security of the underlying FHE scheme (BFV or CKKS). Both schemes are provably secure under the Ring Learning With Errors (R-LWE) assumption. The encryption algorithm is probabilistic, so multiple encryptions of the same location produce different, uncorrelated ciphertexts. A polynomial-time adversary's advantage in distinguishing C_0 and C_1 is negligible in the security parameter λ (128 bits in our implementation).

[Computation Privacy] The server learns no information about the EV's location or the query results during homomorphic evaluation, beyond what is implied by the ciphertexts themselves.

[Proof Sketch] All server-side computation is performed on ciphertexts without access to the secret key sk . The homomorphic evaluation algorithms are deterministic functions of the input ciphertexts and the public key. Since the input ciphertexts are IND-CPA secure, any information the server could learn from the computation would imply an attack on the IND-CPA security of the scheme. Therefore, the server's view during computation is computationally indistinguishable from a view where the ciphertexts are replaced with encryptions of random plaintexts.

B. Declared Leakage

Our protocol explicitly leaks the following information by design:

- 1) **ZoneID (if sharding is used):** The client reveals which coarse geographic tile they are in. This leaks approximately $\log_2(\text{Area}_{\text{total}}/\text{Area}_{\text{tile}})$ bits of information.
- 2) **Query Timing:** The server observes when queries are made, which could reveal information about driving patterns. This is inherent to any real-time service and can be mitigated by dummy queries.
- 3) **Number of Stations Processed:** The server knows how many stations were in the requested ZoneID. This is public information anyway, as the station database is public.
- 4) **Ciphertext Metadata:** The size of ciphertexts and the number of batches processed are fixed for a given ZoneID and parameter set, so no information is leaked through these channels.

C. Security Properties Comparison

TABLE VII
SECURITY PROPERTIES COMPARISON

Property	Our	SC	DP	PIR
IND-CPA Security	✓			✓
Hides Precise Location	✓	Partial	✓	✓
Hides Coarse Region	(ZoneID)	Partial	✓	✓
No Query Accuracy Loss	✓(BFV)			✓
Resists Correlation Attacks	✓			✓
Post-Quantum Secure	✓	N/A	N/A	

VIII. CONCLUSION AND FUTURE WORK

This paper demonstrates that client-optimized Fully Homomorphic Encryption is a viable tool for practical privacy in intelligent transportation systems. By architecting a protocol with depth-0 distance calculation, SIMD batching, and geographic sharding, we enable anonymous EV charging station discovery with end-to-end latencies under 2 seconds for 10,000 stations and client energy consumption below 2 Joules per query. Our reproducible implementation in SecretFlow and comprehensive parameter disclosure address the reproducibility concerns often found in applied cryptographic work.

A. Future Work

- 1) **Embedded Implementation:** Port the protocol to automotive-grade System-on-Modules for real-world road testing.
- 2) **Malicious Security:** Extend the protocol with lightweight verifiable computation techniques to protect against malicious service providers.
- 3) **Complex Query Support:** Support richer queries beyond simple proximity, such as filtering by charger type, pricing, and availability.
- 4) **Hybrid Privacy Models:** Combine FHE with differential privacy for multi-query scenarios.
- 5) **Standardization:** Work with automotive standards bodies to incorporate privacy-preserving computation into emerging standards.

This work provides a clear pathway toward scalable, privacy-preserving vehicular networks where strong cryptographic guarantees and practical performance coexist.

REFERENCES

- [1] International Energy Agency, "Global EV Outlook 2023," IEA Publications, Paris, 2023.
- [2] J. Krumm, "Inference attacks on location tracks," in *Proc. International Conference on Pervasive Computing*, 2007, pp. 127-143.
- [3] Y.-A. de Montjoye, C. A. Hidalgo, M. Verleysen, and V. D. Blondel, "Unique in the crowd: The privacy bounds of human mobility," *Scientific Reports*, vol. 3, no. 1, pp. 1-5, 2013.
- [4] H. Zang and J. Bolot, "Anonymization of location data does not work: A large-scale measurement study," in *Proc. ACM MobiCom*, 2011, pp. 145-156.
- [5] M. Gruteser and D. Grunwald, "Anonymous usage of location-based services through spatial and temporal cloaking," in *Proc. ACM MobiSys*, 2003, pp. 31-42.
- [6] B. Gedik and L. Liu, "Protecting location privacy with personalized k-anonymity: Architecture and algorithms," *IEEE Transactions on Mobile Computing*, vol. 7, no. 1, pp. 1-18, 2008.
- [7] C. Dwork, "Differential privacy," in *Proc. International Colloquium on Automata, Languages and Programming*, 2006, pp. 1-12.
- [8] F. McKeen et al., "Innovative instructions and software model for isolated execution," in *Proc. HASP Workshop*, 2013, pp. 1-8.
- [9] Y. Xu, W. Cui, and M. Peinado, "Controlled-channel attacks: Deterministic side channels for untrusted operating systems," in *Proc. IEEE Symposium on Security and Privacy*, 2015, pp. 640-656.
- [10] B. Chor, O. Goldreich, E. Kushilevitz, and M. Sudan, "Private information retrieval," in *Proc. IEEE FOCS*, 1995, pp. 41-50.
- [11] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proc. ACM STOC*, 2009, pp. 169-178.
- [12] SecretFlow Team, "SecretFlow: A unified framework for privacy-preserving computing," *arXiv preprint arXiv:2308.14310*, 2023.
- [13] K. Chatzikokolakis, C. Palamidessi, and M. Stronati, "A predictive differentially-private mechanism for mobility traces," in *Proc. International Symposium on Privacy Enhancing Technologies*, 2014, pp. 21-41.
- [14] ARM, "ARM Security Technology: Building a Secure System using TrustZone Technology," ARM White Paper, 2009.
- [15] S. Arnaudov et al., "SCONE: Secure Linux containers with Intel SGX," in *Proc. USENIX OSDI*, 2016, pp. 689-703.
- [16] R. Bahmani et al., "CURIOUS: A framework for privacy-preserving location-based services using Intel SGX," in *Proc. IEEE CNS*, 2018, pp. 1-9.
- [17] J. Götzfried, M. Eckert, S. Schinzel, and T. Müller, "Cache attacks on Intel SGX," in *Proc. ACM European Workshop on Systems Security*, 2017, pp. 1-6.
- [18] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. ASIACRYPT*, 2017, pp. 409-437.
- [19] Microsoft Research, "Microsoft SEAL (release 4.0)," 2022. [Online]. Available: <https://github.com/microsoft/SEAL>
- [20] OpenFHE Team, "OpenFHE: Open-source fully homomorphic encryption library," 2022. [Online]. Available: <https://www.openfhe.org>
- [21] A. Abadi, S. Terzis, and C. Dong, "Feather: Lightweight multi-party updatable delegated private set intersection," *IACR Cryptology ePrint Archive*, Report 2020/407, 2020.
- [22] M. A. El-Fotouh, "Homomorphic encryption for navigation applications," *International Journal of Computer Applications*, vol. 182, no. 4, pp. 1-8, 2018.
- [23] Y. Zhang, L. T. Yang, S. Chen, and X. Chen, "Privacy-preserving ride matching using homomorphic encryption in IoT-assisted vehicular fog computing," *IEEE Internet of Things Journal*, vol. 8, no. 4, pp. 2482-2492, 2021.
- [24] Texas Instruments, "Jacinto 7 Processor Family for Automotive Applications," TI Datasheet, 2021.
- [25] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "TFHE: Fast fully homomorphic encryption over the torus," *Journal of Cryptology*, vol. 33, no. 1, pp. 34-91, 2020.